# 标准版 SDK 使用说明文档

金华市蓝海光电技术有限公司

# 目录

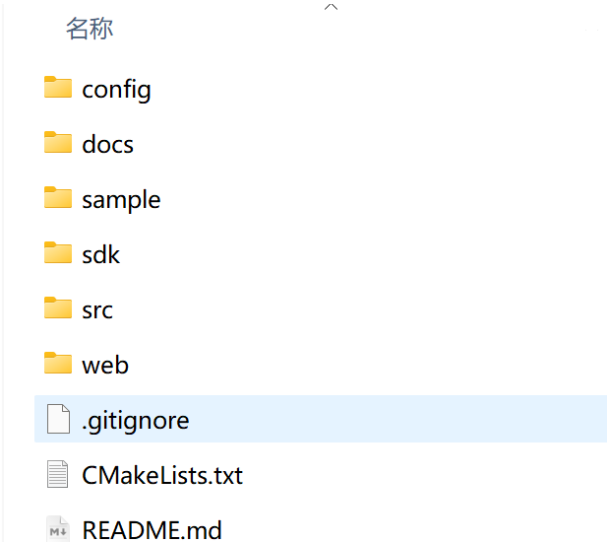
**标准版 SDK 使用说明文档** ..... 1
1. 简介 ..... 1
2. SDK 文件组织 ..... 1
3. SDK 开发指南 ..... 1
3.1 SDK 集成说明 ..... 1
3.2 头文件介绍 ..... 2
3.3 SDK 主要函数接口说明 ..... 2
3.4 注意事项 ..... 7
4. 配置文件说明 ..... 7
5. SDK 编译 ..... 9
6. 示例程序 ..... 10
7. 多雷达配置 ..... 11
8. 控制指令表 ..... 11
9. 异常情况判定 ..... 11
10. 丢包录制功能 ..... 12

# 1. 简介

本文档针对标准开源版本的 LIDAR SDK。目前该 SDK 可以在 Windows、和 Linux 环境下使用。支持 VC++6.0 以上版本(g++ 98/gcc99)编译。

# 2. SDK 文件组织

SDK 的文件结构如下图所示（cmake 工程)：




Config:常用型号的雷达配置文件参数

Docs:文档目录

sample:测试例子，这里 windows/linux 通用

Sdk:核心代码

Src:demo 例子

CMakeLists.txt

Readme.md

# 3. SDK 开发指南

## 3.1 SDK 集成说明

这里以源码的方式提供，集成时需要添加 sdk 文件夹，引入单头文件 standard_interface.h，详细参考 demo 文件夹下的 main.cpp 文件。

点云数据读取、同步时间戳打印、运行信息打印等可以参考

void CallBackMsg(void* context, int msgtype, void *param, int length)函数

参数 1：用户上下文指针

参数 2：自定义的信息类型

参数 3：回调的数据

参数 4：回调的数据长度

主要步骤说明：

1. 主线程通过 addLidarByPath 将配置文件中的数据添加，并返回唯一的 ID
2. 主线程通过 setCallBackPtr 设置每个雷达工作线程的回调函数，参考 CallBackMsg
3. 主线程通过 openDev 打开指定 ID 的雷达，并启动工作线程(必须)
4. 工作线程通过回调函数返回点云的具体信息
5. 主线程通过调用 standard_interface.h 中的接口读取/设置其他功能，详情参考以下的接口

## 3.2 头文件介绍

数据结构：data.h

错误定义头文件：error.h

通用接口：Global.h

数据处理：LidarDataProcess.h

丢包日志记录：managerlog.h

串口单独的函数（linux）Uart.c

对外提供：

standard_interface.h 提供给用户使用的 SDK 头

## 3.3 SDK 主要函数接口说明

这里主要解析 standard_interface.h/cpp 文件，该文件直接提供给客户集成使用。

- addLidarByPath

| | |
|---|---|
| 函数名称： | int addLidarByPath(const char *cfg_file_name) |
| 函数参数： | 1.配置文件的绝对路径+名称 [const char*,IN] |
| 函数作用： | 通过配置文件新增雷达 |
| 返回值： | 雷达 ID |
| 其他说明： | |

- delLidarByID

| | |
|---|---|
| 函数名称： | bool delLidarByID(int ID); |

| | |
|---|---|
| 函数参数： | 1.雷达 ID [int,IN] |
| 函数作用： | 通过 ID 删除指定雷达 |
| 返回值： | True/false |
| 其他说明： | |

- getLidar

| | |
|---|---|
| 函数名称： | `RunConfig *getLidar(int ID);` |
| 函数参数： | 1.雷达 ID [int,IN] |
| 函数作用： | 获取当前雷达的信息 |
| 返回值： | 雷达 ID |
| 其他说明： | |

- setCallBackPtr

| | |
|---|---|
| 函数名称： | `void setCallBackPtr(int ID, printfMsg ptr, void *context);` |
| 函数参数： | 1.雷达 ID [int,IN]<br>2.回调指针 [IN] `typedef void (*printfMsg)(int, void*, void*, int);`<br>3.设备上下文，用户数据指针 |
| 函数作用： | 根据雷达 ID，设置对应的回调指针 |
| 返回值： | |
| 其他说明： | |

- openDev

| | |
|---|---|
| 函数名称： | `bool openDev(int ID);` |
| 函数参数： | 1.雷达 ID [int,IN] |
| 函数作用： | 打开雷达设备，并且运行子线程单独处理数据 |
| 返回值： | True/false |
| 其他说明： | 如果成功运行，将会打印成功信息 |

- StopDev

| | |
|---|---|
| 函数名称： | `void StopDev(int ID);` |
| 函数参数： | 1.雷达 ID [int,IN] |
| 函数作用： | 关闭雷达设备及其子线程 |
| 返回值： | |
| 其他说明： | |

- GetDevInfo

| | |
|---|---|
| 函数名称： | `bool GetDevInfo(int ID, EEpromV101 *eepromv101)` |
| 函数参数： | 1.雷达 ID [int,IN] |

| | |
|---|---|
| | 2.雷达参数信息 [EEpromV101,OUT] |
| 函数作用： | 获取雷达全局参数 |
| 返回值： | True/false |
| 其他说明： | |

- ControlDrv

| | |
|---|---|
| 函数名称： | `bool ControlDrv(int ID, int num, char *cmd)` |
| 函数参数： | 1.雷达 ID [int,IN]<br>2.指令长度 [int,IN]<br>3.指令内容 [char*,IN] |
| 函数作用： | 设置雷达开启或者暂停 |
| 返回值： | True/false |
| 其他说明： | |

- getVersion

| | |
|---|---|
| 函数名称： | `const char *getVersion()` |
| 函数参数： | |
| 函数作用： | 获取版本号 |
| 返回值： | const char* |
| 其他说明： | |

- ZoneSection

| | |
|---|---|
| 函数名称： | `bool ZoneSection(int ID, char section)` |
| 函数参数： | 1.雷达 ID [int,IN]<br>2.激活的防区序号 [char,IN] |
| 函数作用： | 切换当前防区 |
| 返回值： | True/false |
| 其他说明： | |

- SetUDP

| | |
|---|---|
| 函数名称： | `bool SetUDP(int ID, char *ip, char *mask, char *gateway, int port)` |
| 函数参数： | 1.雷达 ID [int,IN]<br>2.ip 地址 [char*,IN]<br>3.子网掩码 [char*,IN]<br>4.网关 [char*,IN]<br>5.端口 [int,IN] |
| 函数作用： | 设置雷达的网络参数 |
| 返回值： | true/false |
| 其他说明： | |

- SetDST

| | |
|---|---|
| 函数名称： | bool SetDST(int ID, char *ip, int port) |
| 函数参数： | 1.雷达 ID [int,IN]<br>2.ip 地址 [char*,IN]<br>5.端口 [int,IN] |
| 函数作用： | 设置雷达的上传网络参数地址 |
| 返回值： | true/false |
| 其他说明： | |

- SetRPM

| | |
|---|---|
| 函数名称： | bool SetRPM(int ID, int RPM) |
| 函数参数： | 1.雷达 ID [int,IN]<br>2.转速 [int,IN] |
| 函数作用： | 设置雷达转速 |
| 返回值： | true/false |
| 其他说明： | |

- SetTFX

| | |
|---|---|
| 函数名称： | bool SetTFX(int ID, bool tfx) |
| 函数参数： | 1.雷达 ID [int,IN]<br>2.tfx [bool,IN] |
| 函数作用： | 设置雷达固定上传 |
| 返回值： | true/false |
| 其他说明： | |

- SetDSW

| | |
|---|---|
| 函数名称： | bool SetDSW(int ID, bool dsw) |
| 函数参数： | 1.雷达 ID [int,IN]<br>2.dsw [bool,IN] |
| 函数作用： | 设置雷达滤波，去拖点 |
| 返回值： | true/false |
| 其他说明： | |

- SetSMT

| | |
|---|---|
| 函数名称： | bool SetSMT(int ID, bool smt) |
| 函数参数： | 1.雷达 ID [int,IN]<br>2.smt [bool,IN] |

| | |
|---|---|
| 函数作用： | 设置雷达数据平滑 |
| 返回值： | true/false |
| 其他说明： | |

- SetPST

| | |
|---|---|
| 函数名称： | bool SetPST(int ID, int mode) |
| 函数参数： | 1.雷达 ID [int,IN]<br>2.mode [int,IN]　0 无数据 1 仅数据上传 2 仅报警 3 数据+报警 |
| 函数作用： | 设置雷达上传方式 |
| 返回值： | true/false |
| 其他说明： | |

- SetDID

| | |
|---|---|
| 函数名称： | bool SetDID(int ID, unsigned int number) |
| 函数参数： | 1.雷达 ID [int,IN]<br>2.雷达的编号 [unsigned int,IN] |
| 函数作用： | 设置雷达编号 |
| 返回值： | true/false |
| 其他说明： | |

- SetNTP

| | |
|---|---|
| 函数名称： | bool SetNTP(int ID, char *ntp_ip, unsigned int port, bool enable) |
| 函数参数： | 1.雷达 ID [int,IN]<br>2.雷达的 ntp ip [char*,IN]<br>3.雷达的 ntp 上传端口 [unsigned int,IN]<br>4.使能 [bool,IN] |
| 函数作用： | 设置雷达 ntp 参数 |
| 返回值： | true/false |
| 其他说明： | |

- SetPTP

| | |
|---|---|
| 函数名称： | bool SetPTP(int ID, bool enable) |
| 函数参数： | 1.雷达 ID [int,IN]<br>2.使能 [bool,IN] |
| 函数作用： | 设置雷达 ptp |
| 返回值： | true/false |
| 其他说明： | |

- SetRES

| | |
|---|---|
| 函数名称： | `bool SetRES(int ID, unsigned int res)` |
| 函数参数： | 1.雷达 ID [int,IN]<br>2.res_value [char*,IN]<br>100：0.1°(600 rpm)/0.15(900 rpm)；200：0.2°(600 rpm)/0.3(900 rpm) |
| 函数作用： | 设置雷达角分辨率 |
| 返回值： | true/false |
| 其他说明： | |

- SetRecordConfig

| | |
|---|---|
| 函数名称： | `void SetRecordConfig(bool enabled, const std::string& root_path = "", uint64_t min_disk_mb = 100, uint64_t max_queue_bytes = 4096);` |
| 函数参数： | 1.enabled [bool,IN] 是否开启录制<br>2.root_path [const std::string&,IN] 录制数据保存的根目录<br>3.min_disk_mb [uint64_t,IN] 录制停止剩余空间阈值<br>4.max_queue_bytes[uint64_t,IN] 日志队列最大字节数 |
| 函数作用： | 设置数据录制功能 |
| 返回值： | |
| 其他说明： | 当前仅支持一个雷达 |

### 3.4 注意事项

SDK 使用 C/C++方式开发。建议开发人员在使用 SDK 前，对雷达的通讯协议(uart/udp)和工作模式(硬件正常工作的情况，包括指示灯等)有所了解。可以参考具体型号的使用手册获取相关细节。

配置文件默认 utf-8 的格式，如果被设置为 gbk，在 linux 下使用会出现问题，以下为转换方式：因为平台间回车换行符的问题，可以通过 vim 打开 xxx.txt 文件，set ff=unix 来一键修改文件格式。

## 4. 配置文件说明

特殊说明:有些雷达仅支持部分参数，具体参考 config 目录配置文件

| connect | | |
|---|---|---|
| type | 雷达的类型 | uart：串口 |
| | | udp：网络款 |
| | | udp_uart：虚拟串口 |
| connectArg | 串口雷达的 USB 口号 | Win:com1<br>Linux:/dev/ttyUSB0 |

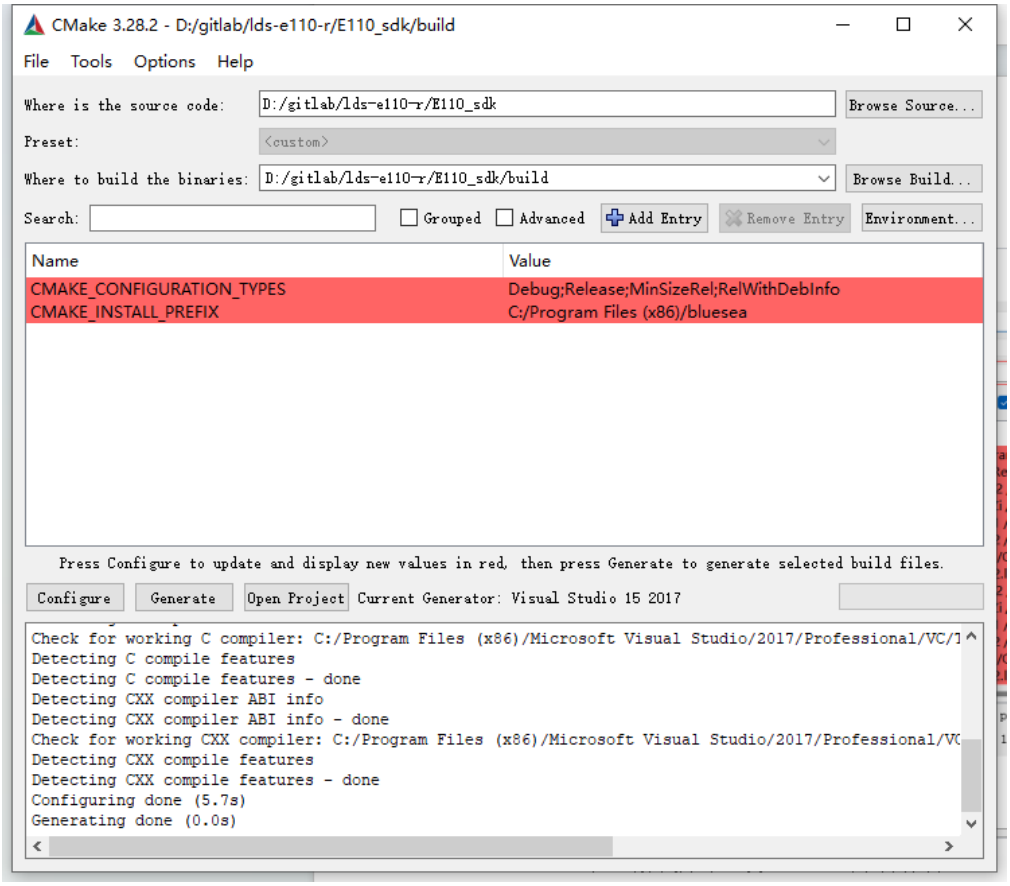
| | | |
|---|---|---|
| | | Linux VPC:/dev/ttyACM0 |
| | 目标雷达的 IP | 举例：192.168.158.98 |
| connectArg2 | 串口雷达的波特率 | 举例：768000 |
| | 目标雷达的访问端口 | 举例：6543 |
| local_port | 主机的接收端口 | 举例：6888 |
| is_group_listener | 是否开启广播 | 0/1 |
| group_ip | 广播 IP | 224.0.0.99<br>需要在雷达上位机里修改上传 IP 开启 |
| data | | |
| data_bytes | 打包字节数 | 3 |
| with_checksum | 是否校验 | 0/1 |
| output_360 | 是否按 360 度一次输出 | 0：部分扇区 1：全扇形 |
| from_zero | 是否零度输出 | 0/1 0 -180°-180° 1 0°-360 |
| service_port | Web 本都服务的开放端口 | 举例：8888 |
| is_open_service | 是否启动 web 服务 | 0/1 |
| error_circle | 统计符合要求（距离为 0 的点数量/总数大于指定系数）的错误圈数 | 默认 3 |
| error_scale | 比例系数 距离为 0 的点:总点数 | 默认 0.9 |
| get | | |
| uuid | 查询 sn | -1 不执行 >=0 执行 |
| model | 查询型号 | -1 不执行 >=0 执行 |
| version | 查询硬件版本号 | -1 不执行 >=0 执行 |
| set | | |
| unit_is_mm | 是否是毫米为数据单位 | -1/0/1 不执行/否/是 |
| with_confidence | 是否数据带强度 | -1/0/1 不执行/否/是 |
| with_smooth | 是否数据平滑 | -1/0/1 不执行/否/是 |
| with_deshadow | 是否去拖点 | -1/0/1 不执行/否/是 |
| resample | 角分辨率 | -1/0/1/xxx 不执行/非固定/固定角分辨率/具体的固定值 |
| rpm | 转速 | -1/xx 不执行/具体转速 |
| direction | 旋转方向 | -1/0/1 不执行/顺时针/逆时针 仅部分雷达生效 |
| alarm_msg | 报警信息上传 | -1/0/1 不执行/仅数据/数据加报警 |
| ats | 连接方式 | -1/0/1 不执行/网络/USB |

| | | |
|---|---|---|
| with_start | 启动时默认发送开始旋转指令 | -1/0/1　不执行/否/是 |

# 5. SDK 编译

Windows:

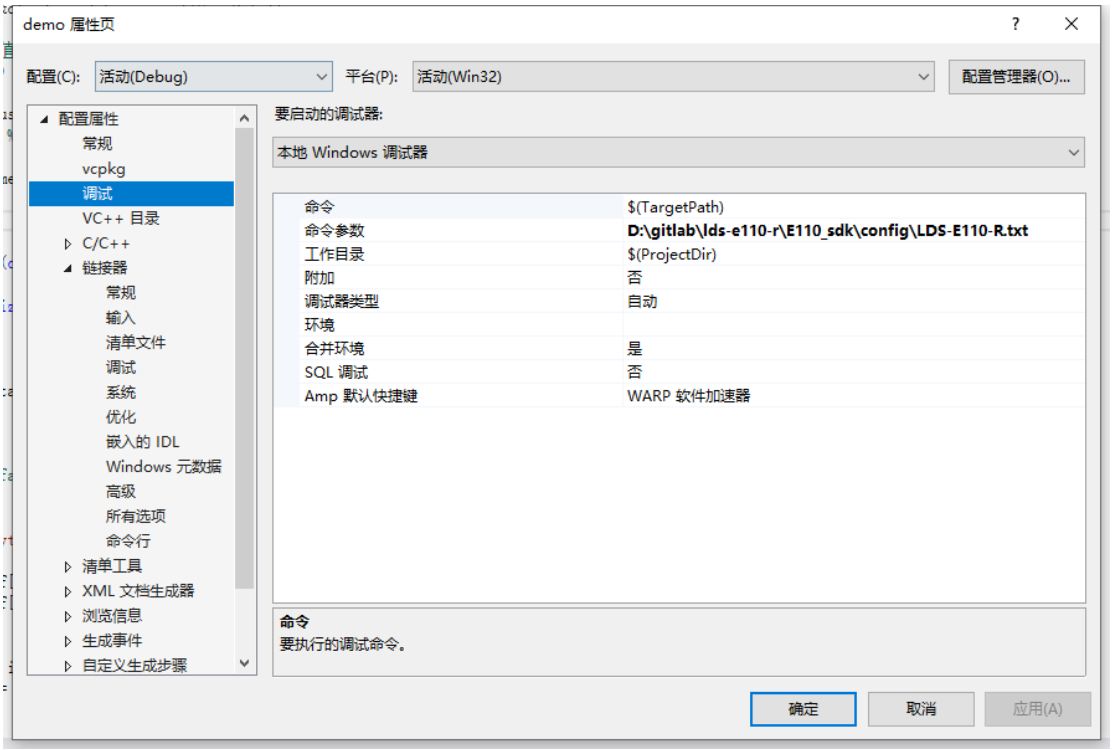
1.需要下载 cmake 工具，然后使用 cmake-gui 生成指定的 vs 工程，这里以 vs2022 为例。




2.vs 工程：解决方案->设置 demo 为启动项

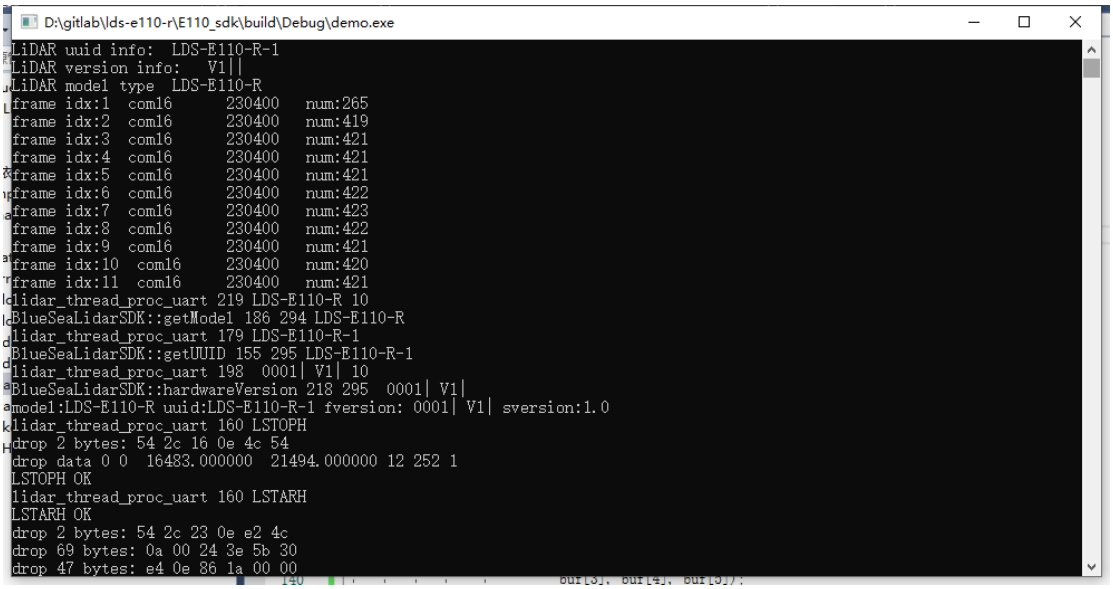
3.项目->属性->调试，设置命令参数(调用雷达的配置文件)以及工作目录(固定)。

或者直接在命令参数写入配置文件的绝对路径




4.生成->生成解决方案

5.调试->开始运行




Linux

cmake CMakeList.txt

make

./demo config/xxxx.txt

注:如果是虚拟机下的串口设备，还需要 sudo chmod 777 /dev/ttyUSB*,给设备赋权

# 6. 示例程序

源码详情参考 src 目录下的 main.cpp 文件

# 7. 多雷达配置

这里将配置文件的相对路径(绝对路径)传入，
例如 ./demo config/LDS-E110-R.txt config/LDS-E110-R-2.txt 具体配置信息如下




注意：成功运行后，会有两个运行成功打印，如果有一个失败，就会直接退出进程

# 8. 控制指令表

说明：这里指 SDK 中涉及到的硬件指令含义,并不是所有型号都全部支持以下命令，具体型号支持的指令查看该雷达型号的用户手册。

| 指令 | 说明 |
|---|---|
| LSTARH | 开始运行 |
| LSTOPH | 停止运行 |
| LRESTH | 重新运行(重新上下电) |
| LUUIDH | 查询 SN 号 |
| LVERSH | 获取版本信息 |
| LTYPEH | 获取具体型号 |

# 9. 异常情况判定

1.Sdk 中的 LidarCheckService 类，可以对雷达的在线离线状态进行判定，如果雷达在 2s 内没收到心跳包广播，则会显示离线。

```
Please control it through a browser and enter the default address: http://localhost: 8888
send command : 'LVERSH'
set LiDAR LVERSH  OK V3.0

ntp ip set error!
info: Lidar start work,first span angle is 180
SDK version:3.2.4
heart:192.168.0.147 online
heart:192.168.0.147 offline
udp no valid data, count:1, 2026-01-31 15:30:09.077
udp no valid data, GetDevInfo_MSG failed
udp no valid data, count:2, 2026-01-31 15:30:11.093
udp no valid data, GetDevInfo_MSG failed
udp no valid data, count:3, 2026-01-31 15:30:13.098
udp no valid data, GetDevInfo_MSG failed
udp no valid data, count:4, 2026-01-31 15:30:15.114
udp no valid data, GetDevInfo_MSG failed
udp no valid data, count:5, 2026-01-31 15:30:17.126
udp no valid data, GetDevInfo_MSG failed
error: 1769844618 809 span err  code:-1 value:angle sum 1710, drop 19 fans 1425 points
heart:192.168.0.147 online
```

2.Sdk 中在 lidar_thread_proc_uart 和 lidar_thread_proc_udp 中对是否收到有效数据进行了判断，如果在 1s 内没有收到有效数据，则会进行输出，并发送 uuid 命令尝试和雷达通信。

```
Please control it through a browser and enter the default address: http://localhost: 8888
send command : 'LVERSH'
set LiDAR LVERSH  OK V3.0

ntp ip set error!
info: Lidar start work,first span angle is 180
SDK version:3.2.4
heart:192.168.0.147 online
heart:192.168.0.147 offline
udp no valid data, count:1, 2026-01-31 15:30:09.077
udp no valid data, GetDevInfo_MSG failed
udp no valid data, count:2, 2026-01-31 15:30:11.093
udp no valid data, GetDevInfo_MSG failed
udp no valid data, count:3, 2026-01-31 15:30:13.098
udp no valid data, GetDevInfo_MSG failed
udp no valid data, count:4, 2026-01-31 15:30:15.114
udp no valid data, GetDevInfo_MSG failed
udp no valid data, count:5, 2026-01-31 15:30:17.126
udp no valid data, GetDevInfo_MSG failed
error: 1769844618 809 span err  code:-1 value:angle sum 1710, drop 19 fans 1425 points
heart:192.168.0.147 online
```

# 10. 丢包录制功能

在雷达设备成功初始化后，若调用 SetRecordConfig(true) 启用录制功能，SDK 将进入丢包事件触发式录制模式。

在此模式下，SDK 会持续缓存通过串口或 UDP 接收的原始数据流。当检测到某一帧数据不完整时，系统将自动触发录制流程，并将该异常帧前后一定时间窗口内的二进制数据写入磁盘文件。

录制将持续进行，直至数据流恢复正常并保持稳定达 3 秒，随后自动停止。若后续再次发生丢包事件，系统将重复上述录制过程。

所生成的日志文件可用于分析网络或通信链路中的异常原因，辅助定位丢包根源。

